

I capacity of the teams.

Hands on: Play with boards

- Todd's CMO board is a group level board, where labels define the lists:
<https://gitlab.com/groups/gitlab-com/marketing/-/boards/906226>
 - The Product and Solution Marketing Sales Enablement board: <https://gitlab.com/gitlab-com/marketing/product-marketing/boards/918030>
 - The Plan/ Milestone board, where the lists are defined by Milestones:
<https://gitlab.com/groups/gitlab-org/-/boards/706864>
 - The Plan Backend assignment board, where the lists are defined by who will be doing the work on a specific release: <https://gitlab.com/groups/gitlab-org/-/boards/631316>
- Create an BOARD in this Project : Account Based Marketing

Milestones

Milestones establish a target date for a sprint or specific bundle of issues and code changes to be delivered. The milestone enables the team to either set a specific Start and Stop for the work, as in a Sprint, or the milestone could be a fixed point in time. Milestones are EITHER Group or Project

| Milestones in GitLab are a way to track issues and merge requests created to achieve a broader goal in a certain period of time. Milestones allow you to organize issues and merge requests into a cohesive group, with an optional start date and an optional due date. |
..... |

Group Level Planning

Epics

In order to track groups of related issues, the GitLab epic gives product owners and leaders the ability to link and manage work over an extended time frame. Planning work use Epics allows you to create parent and child epics. An epic can span multiple milestones and makes it easier to manage the overall flow and priority of work. Epics are GROUP ONLY

| Epics let you manage your portfolio of projects more efficiently and with less effort by tracking groups of issues that share a theme, across projects and milestones | |

Roadmaps

| The roadmap is a visual representation of the various epics for the group. The roadmap view can be filtered by label and organized by start / stop date of the epics in order to visualize the sequence of work. At this point, GitLab doesn't create dependencies between issues or epics. | |

SECTION: marketing/brand-and-product-marketing/product-and-solution-marketing/getting-

started/103.md

Objective

Marketing creates and maintains quite a few presentations for various stakeholders - like analysts, sales, customers. Some of the commonly maintained presentations are available in the sales resources handbook page. There is common content across the slides. The focus of this page is to explain how we manage common slides across various presentations that strategic marketing creates for various stakeholders.

Single directory with commonly used slides

All common slides will be maintained as individual slides in this strategic marketing master slides folder. The most recent and up to date version of the common slides will be maintained in this folder.

How to link common slides to your presentation

Description	Image
--- ---	
Open and copy the common slide you need from the master slides folder	
Paste and link common slide in destination presentation	
(Optional) Make changes in source common slide	
Any changes to the source slide shows up as an Update in the destination presentation. If you would like to update your presentation, click on Update	
Tada! View the updates in the destination presentation	

SECTION: marketing/brand-and-product-marketing/product-and-solution-marketing/getting-started/104.md

Objective

Often you need to manage a set of consistently repeating events, where multiple issues are used to track the combined activity of a team. This page describes how to utilize GitLab Issue Import and a few spreadsheet tricks to make it possible to create a repeatable set of issues for future projects.

The Issue Import CSV button is at the top of the GitLab project issue list. This feature can be very useful for repeating projects with similar groups of issues / work.

Please read the "Tips and Tricks" section before you import issues into your projects

Assumptions

1. You're familiar with the GitLab structure of how projects, groups, epics, milestones, issues, and labels work. If not, please review GitLab 101- No Tissues with Issues

2. You're comfortable working in a spreadsheet and using formulas to concatenate text into a cell. note: In my examples, I use the & operator, which is the same as the concat function.

3. You have Developer permission level or higher in your project. (required to do issue import)

4. Your issues will be created in the same project. You can still do this for cross-project templates, but you will either need to MOVE issues to the destination project (can be automated) or more manually you would need to upload a separate CSV for each individual project. Check out this automation tutorial for how to automate moving an issue from one project to another.

Steps

1. Build Issue spreadsheet

Decide what project structure will be for your set of issues. Specifically,

- what labels will be used for the project,
- what epics and sub-epics will be used to track the work, and
- what milestones will you use for this work effort.

If these structures do not already exist in GitLab, configure them now.

1.1 Spreadsheet setup

1. While you only really need two columns, (Title, and Description), I've found it's easier to have several columns so you can keep details separate and manage issues. Configure a spreadsheet with the following columns

First: - Two columns where the BASIC text of the issue title and description go.

- raw title,
- raw description,

These two columns is where you will enter the text for the titles and descriptions of your issues.

Second: Two columns where the actual text for the issue title and description are 'assembled'

- title,
- description

These two columns are actually just formulas that combine different cells to make up the Title and Description for your issues.

Third: One or more additional columns to hold issue specific QUICK ACTIONS. In these columns, you can add content such as; epics, estimates, labels, and more can be stored. In the example spreadsheet these columns each contain specific data for each issue:

Team Label, Type, Effort, Size, Epic, Milestone, Iteration

When you're done it will look like this:

Fourth: If you want to have the same Label or the same Quick Action for ALL the issues. A cell at the top of the spreadsheet is a place for common Quick Actions

1.2 Draft your first Issue

1. In the row below the headings

1.1 - Raw Title - Write the text for the issue title

1.2 - Raw Description - Using markdown, write the issue description.

Note: it's often much easier to work in the Formula bar in google sheets, which makes it easier to enter multi line text in the cell.

2. In the Quick Action columns, add the appropriate quick actions to add labels, set efforts, add to milestones or epics. This is where you can customize each issue to be tracked in GitLab.

Note: If you are using automation, you could use a label to promote the issue to be an epic or a label to automate moving the issue to another project, to make it confidential, or whatever you want to do with quick actions. Read this automation tutorial to learn more about how automation can update issues for you.

See this publicly available example import spreadsheet.

2. Import Your Issues

1. Select the title and description cells (including the column headers) and copy the contents.
2. Go to a new empty spreadsheet and paste as values. This will eliminate the formulas, but will keep the actual values.
3. Save the new spreadsheet as a CSV file. The file should only be the two columns title and description
4. Go to your project issue list and click on the Import CSV button:
5. In the dialogue box, click on the file name button and select the CSV file.
6. Click the Import Issues button and wait for the magic to happen.

IMPORTANT - Tips, Tricks, and Hints - READ THIS FIRST

1. Create a separate project to TEST your import file. There is NO WAY TO DELETE ISSUES in

GitLab. You can CLOSE issues, but you cannot delete them. So, if you are importing 20 issues and you make a mistake, depending on where your test project is in your GitLab hierarchy, you will need to:

- Remove all the labels, milestones, epics, etc (labels and epics roll up to higher level groups)
- Un-assign any issues that were assigned to individuals
- Close the issues

1. Use a real issue in your GitLab project to write your Quick Action codes. Then, copy and paste the quick actions to your spreadsheet. That way your quick actions will not have typos or errors.

1. The Issue ORDER will be sequential starting at the first row then working to the last. That means your first row in the your spreadsheet will show up LAST in GitLab (when sorted by the default issue created date). However, if you list your spreadsheet in the reverse order, then your last issues will be loaded first. That means they will be OLDER, and therefore listed last in GitLab issue lists.

SECTION: marketing/brand-and-product-marketing/product-and-solution-marketing/getting-started/105.md

Objective

When you use issues and labels to manage day to day work and define a specific process, often there are situations and times where it's hard to keep labels in the right status. For example:

- Label Hygiene: Consider a situation where there is a process defined by an overall label (ABC) and the specific steps in the process are defined by the scoped labels (ABC::1, ABC::2, and ABC::3)
- The process assumes that everyone in the team will assign "ABC" and one of the scoped labels "ABC::x"
- However it's likely that people will forget to assign both labels, and over time there will be a mixture of issues missing one of the two labels.

- Label Hygiene Consider a process, where you want to summarize issues based on a set of detailed labels that might be assigned to them. Such as, a process where there are 10 scoped labels "ZYX::1, ZYX::2, ZYX::3, ... ZYX::10", and you want to summarize them in to "ODD" and "EVEN". So ZYX::2,4,6,8,10 all would be labeled "EVEN" and ZYX::1,3,5,7,9 would be labeled "ODD".

- You could request your team to update their issues with the right "ODD"/"EVEN" labels based on the ZYX::nnn label
- You could bulk update labels (find all the ZYX::1 issues and then add the "ODD" label, and repeat 9 more times)

In either case, the ODD/EVEN summary labels would be most likely incorrect, as people would forget to update them, or if something changed, they would be wrong

There are other key situations where automation can help. (see below for these practical

examples)

- Moving issues to a certain project - consider a situation where you have an issue or group of issues that you want to periodically move to a specific project.
- Promoting issues to be epics - consider a situation where you have a set of related epics you want to re-create. You have the epics defined as issue templates and want them to automatically be promoted to be epics.
- Deprecating/deleting an issue - Consider where you have an issue that you want to effectively delete and therefore remove all the labels, epics, estimates, milestones etc.

Automation can address all of these situations and streamline work.

Solution - Label Hygiene / Triage Bot

Define a set of rules that will automatically

- Add a label when it is missing
- Change or remove a label when it's wrong
- Add a comment, mention a user, and more thorough comments and /quick actions.

The Triage Bot is an Open GitLab project that makes it possible to automate many issue and merge request hygiene tasks to ensure that your projects are more consistently following a given process.

Example Triage Policies/rules

Here are two examples of how Triage Policies/Rules can automate updates and issue hygiene:

1. This example finds issues that are more than 5 days old, but have no label associated with them, then it adds a comment to ask the Author to do something. The policy also notifies a team member to make them aware of the issue.

yaml

```
- name: My policy
conditions:
  date:
    attribute: updatedat
    condition: olderthan
    intervaltype: days
    interval: 5
  state: opened
  labels:
    - No Label
limits:
  mostrecent: 50
actions:
  labels:
    - needs attention
  mention:
    - markglenfletcher
```

```
comment: |
  {{author}} This issue is unlabeled after 5 days. It needs attention. Please take
care of this before the end of {2.days.fromnow.strftime('%Y-%m-%d')}
```

2. In Strategic marketing, here is a rule that finds any Strategic marketing Requests that are missing a Scoped Workflow label.

yaml

```
- name: smrequest (MISSING a scoped label and assign it as a New Reqeust)
  conditions:
    labels:
      - smrequest
  state: opened
  forbiddenlabels:
    - smreq::triage
    - smreq::transferred
    - smreq::newrequest
    - smreq::declined
    - smreq::completed
    - smreq::backlog
    - smreq::assigned
  actions:
    labels:
      - smreq::newrequest
  comment: |
    SM TriageBot helping out here: This SM Request issue was not in the workflow,
    automatically adding it as a new request.
```

Setting Up the Triage Bot

There are only two simple steps to set up the TriageBot.

1. Configuring your CI/CD pipeline
2. Drafting your first Triage Policy
3. Setting up a CI/CD Schedule to run your Triage Policies periodically.

Step 1: Configuring your CI/CD Pipeline for the first time

1. The triage bot runs as a scheduled CI pipeline job, so your project will need to define a `gitlab-ci.yml` file. The `gitlab-ci.yml` file is stored in the root directory of your project.

Here is an example of the Product and Solution Marketing pipeline, where there are three Jobs defined. There are two Manual jobs, and one Scheduled Job. The two Manual jobs either `test(dry-run)` the rules/policies, or actually apply the rules/policies (`policy:run`). The one Scheduled job (`schedule:policyrun`), when runs will actually apply the rules/policies.

yaml

```
>image: ruby:2.7
```

stages:

- triage
- run

dry-run:trriage:

stage: triage

script:

- gem install gitlab-triage
- gitlab-triage --help
- gitlab-triage --dry-run --token \$APITOKEN --source projects --source-id \$CIPROJECTPATH

when: manual

except:

- schedules

policy:run:

stage: run

script:

- gem install gitlab-triage
- gitlab-triage --token \$APITOKEN --source projects --source-id \$CIPROJECTPATH

when: manual

except:

- schedules

schedule:policyrun:

stage: run

script:

- gem install gitlab-triage
- gitlab-triage --token \$APITOKEN --source projects --source-id \$CIPROJECTPATH

only:

- schedules

2. The TriageBot needs to have access and permission to your GitLab project to read and update issues. This is called a "token", and is stored in GitLab using a "CI/CD Variable" - in this case it is the \$APITOKEN that you see in the above example pipeline.

You will need to configure a GitLab CI/CD Environment Variable\$APITOKEN for these jobs to run.

Here's how:

| 2. Setting Up CI-CD API Token Variable | Image |

|-----|-----|

| 1. In your project select the left menu option Settings-->CI/CD | |

| 2. Then expand the Variables section | |

|3. Add a new line where
 - Type= "Variable"
 - Key = "APITOKEN"
 - Value = Your API Key. - You have an API Key don't you? (See short instructions below)
 - Set Masked to True. | |

| 4. Save Variables | |

| 3. Getting your API Key | Image |

|-----|-----|

- | 1. The API Key is linked to your account and basically gives the Bot permission to act on your behalf. First click on the dropdown on your personal settings in GitLab. | |
- | 2. Select Access Tokens and
 - Give the access token a Name
 - Leave the Expiration Date blank (unless you want it to expire)
 - Select "API" for the Scopes
 - Click Create personal access token | |
- | 3. The Personal Access Token will appear at the top of the page. | |
- | 4. Copy the Token and then Add it to the APITOKEN Value in the CI/CD Variables settings. |
- |

Step 2: Setting up your FIRST Triage policy

The Triage Policies/Rules are defined in a YML file called .triage-policies.yml which is stored in the root directory of your project. For DETAILED instructions about rules and policy options read the Defining a Policy section in the README.md file in the Triage Bot project.

1. Create a new file in the root directory called .triage-policies.yml
1. Paste in the following first simple rule:

```
yml
rules:
  issues:
    rules:
      - name: find all open issues - any label (simple - should be lots)
        conditions:
          state: opened
```

1. Commit and merge your changes.

This will trigger a pipeline, where the Jobs should all be skipped. You should see this at the bottom of your screen.

| 4. Dry-Run TriageBot | Image |

|-----|-----|

- | 1. Click on the Pipeline link and go to the pipeline screen | |
- | 2. Run the "Dry Run" pipeline job. | |
- | 3. Click on the job and watch it run. | |
- | 4. First it starts by creating a container and getting a runner growing (That means your CI job is defined OK so far) | |
- | 5. Then it prompts the Triage bot to show it's help output (that means it's working, and your API key is good) | |
- | 6. Then the job runs the "Dry-Run" and you see the output from the bot as it runs. Just one job - find all the open issues, so it runs fast, finds over 1,000 issues. Since the policy/rule had no filters or actions, the job finishes. | |

Now, you need to define any specific policies/rules that you need for your project.

Step 3: Scheduling Jobs

| Scheduling Jobs | Image |

|-----|-----|

| 1. Go to CI/CD-->Pipelines on the left menu | |

| 2. Click on Create New Schedule | |

| 3. Create the schedule
 - Write a Name/description
 - Select an Interval pattern (how frequently do you want it to run. Probably daily.)
 - Leave the Target Branch, and Variables options with the default
 - Click on Save Pipeline Schedule. | |

Other specific scenarios

Moving issues to a certain project

Create a label in your project that is the flag that tells the automation bot to move the issue.

- The label must be unique for the destination project. For example move--> project2Test

1. Update .triage-policies.yml

1. Paste in the following rule:

yml

resourcerules:

issues:

rules:

- name: 1. Moving an issue to another project based on the label - move to Test

Project 2

conditions:

labels:

- move--> project2Test

state: opened

actions:

comment: |

/move gitlab-com/marketing/strategic-marketing/testgroup-pmm-isights/testproject2

This rule will look for the label move--> project2Test in all open issues and then apply the quick action

- /move where it specifies the destination for the issue.

In the future if you want to move a bunch of issues to that project, all you have to do is add the move--> project2Test label.

Promoting issues to be epics

Create a label in your project that is the flag that tells the automation bot to promote the issue to be an epic.

- For example promote-me

1. Update your .triage-policies.yml
1. Paste in the following rule:

```
yml
resourcerules:
  issues:
    rules:
      - name: 1.2 Promote Issues to epics
        conditions:
          labels:
            - promote-me
          state: opened
        actions:
          comment: |
            /promote
```

This rule will look for the label promote-me in all open issues and then apply the quick action

- /promote which promotes the issue into an epic.

In the future if you want to promote a bunch of issues, all you have to do is add the promote-me label.

Deprecating/deleting an issue

If you want to effectively delete an issue (strip it of labels, remove it from epics and milestones, and remove estimates, and close it)

Create a label in your project that is the flag that tells the automation bot to promote the issue to be an epic.

- For example deprecate-me

1. Update your .triage-policies.yml
1. Paste in the following rule:

```
yml
resourcerules:
  issues:
    rules:
      - name: 1.1 - Deprecate Issues Close and strip labels of deprecated issues
        conditions:
          labels:
            - deprecate-me
          state: opened
        actions:
          comment: |
            /removemilestone
```

```
/removepic  
/removeestimate  
/unlabel  
/title Deprecated Issue  
/close
```

This rule will look for the label deprecate-me in all open issues and then apply a bunch of quick actions that update and close the issue.

In the future if you want to delete and deprecate a bunch of issues, all you have to do is add the deprecate-me label.

Success and Next Steps

From here, you can build/draft your specific policies/rules for your project.

SECTION: marketing/brand-and-product-marketing/product-and-solution-marketing/getting-started/_index.md

Product and Solution Marketing General On-Boarding

- Getting Started 101 - No Tissues with Issues
 - Getting Started 102 - Working at local speed
 - Getting Started 103 - Maintaining common slides across decks
 - Getting Started 104 - Epic Templates - Repeatable sets of issues
 - Getting Started 105 - Label Hygiene - Keeping your issues squeaky clean
 - Keeping Yourself Informed
 - Searching the GitLab Website Like a Pro
- Getting Started xxx - Project management Overview

Teams Specific

Competitive intelligence

Market Research & Customer insight

Partner & Channel Marketing

Product marketing

- Go to the PMM onboarding page and check out our onboarding guide.

Technical marketing

- Creating a Kubernetes Cluster

SECTION: marketing/brand-and-product-marketing/product-and-solution-marketing/getting-started/create-cluster-101.md

Objective: Manually creating a Kubernetes cluster to use with Auto DevOps in GitLab

You will need Kubernetes cluster up and running in order to use GitLab Auto DevOps feature. This is a step by step guide to help you to manually create a Kubernetes cluster in Google Cloud Platform.

More details Auto DevOps

Preparation

You need a Google Cloud Platform account, GitLab employees will have this. Ensure you are logged in with your GitLab account.

Steps to create Kubernetes cluster

- | Step | What it looks like |
- | 1. Open a browser and login to Google Kubernetes Engine |
- | 2. GitLab employees - Sign-in using your GitLab email. | |
- | 3. GitLab employees should use the gitlab-demos project. Others should select or create a project to work in. | |
- | 4. Under COMPUTE on the side bar select Kubernetes Engine -> Clusters. | |
- | 5. Click CREATE CLUSTER. | |
- | 6. Set name for your cluster.
 GitLab Product and Solution Marketing employees, the naming convention is sm\<yourname>, or, if you are in the Product Marketing sub-group use pmm\<yourname>, or for the Technical Marketing group use tmm\<yourname>. Keep the overall name short (up to 16 chars).
Set the number of nodes to 3, and expand the page for additional settings. | |
- | 7. GitLab employees should select the 'demonet' network in Networking. Others should select or create network. | |
- | 8. Enable legacy authorization. | |
- | 9. You are ready to create the cluster, click Create. | |
- | 10. It will take a few minutes to create the cluster, once it is created, you will see your just created cluster on the list. | |
- | 11. Now you are ready to integrate your GitLab project with Kubernetes, and use the Auto DevOps feature.
- | 12. Cleanup steps PLACEHOLDER ||

SECTION: marketing/brand-and-product-marketing/product-and-solution-marketing/getting-started/keeping-yourself-informed.md

Communications within Product and Solution Marketing

As a distributed organization, GitLab and our Product and Solution Marketing department and

teams use asynchronous, transparent, frequent, and multimodal communication to keep ourselves informed.

Our most important communication channels are issues, boards, and GitLab itself, followed by this handbook, Slack, Zoom, and other tools. See the general GitLab Communication page and Tools and Tips for more information.

Below are the parts of GitLab and Slack that are used most in Product and Solution Marketing.

GitLab trackers, boards, handbook, etc

Copy in from /product-marketing.

Slack channels

GitLab has more Slack channels than users at last count, 1,492 public channels for 1,069 weekly active users.

These lists will make it a little easier to get started. They are arranged as general to GitLab, specific to Product and Solution Marketing, then a few more recommendations and some just for fun, before sharing a few Slack pro-tips.

Reminder: Slack data are only retained for 90 days. If you share, discover, or decide something important there, ensure it has a permanent place in our handbook, product docs, etc. and read the general handbook section for Slack guidelines.

GitLab

Read all of

- whats-happening-at-gitlab: usually redundant with company calls but keep up with
- company-fyi: company-wide announcements channel

Highly recommended

- is-this-known: bugs, canary failures
- questions: business and technical questions
- thanks: if they're awesome and you know it, chat your thanks

Product and Solution Marketing

You're up to date if you catch up in GitLab, make meetings or see notes, and depending on your team, catch up with:

- analyst-relations
- competition
- marketing
- pmms
- pmmteam
- product-marketing

- release-post: when shadowing or leading a post
- tech-pmm

Also recommended

- cloud-aws, cloud-gcp, backend, ceo, cfp, content-updates, development for broken master, environment issues, frontend, fundraising, handbook, external-comms for collaboration with our PR agency, incidents, product, production, quality, talent-acquisition, sales, security, supportescalations, UX
- Help yourself! find many more popular and active channels at /statschannels

Have some fun

- dadjokes, office-today, random
- local channels: for your area or one you're visiting, hit command-K or 'Jump to...' in the upper left of the UI and search for 'loc'. From locnyc to locpnw, and from locindia to locberlin there are at least 75 local channels.

Slack hacks

Don't miss the critical handbook section with Slack guidelines. We also recommend:

- use stars: on the upper left of the UI, star the channels and direct message threads you use most
- monitor keywords: go to Slack > preferences and go to 'Notifications'. Scroll down to 'My keywords' and add as many as you like, comma-separated. Keywords some of use include 'PMM' and 'product marketing'.
- tailor notifications: go to Slack > preferences and go to 'Notifications'. Tweak to your heart's content. To set channel-specific notifications, go to the channel and click the channel name in the conversation header. Next, click the notifications drop-down menu and choose your notification preference.
- slack-power-users: of course! a channel for our friendly neighborhood power users. Join at your own risk.

SECTION: marketing/brand-and-product-marketing/product-and-solution-marketing/getting-started/sm-project-management.md

Project management overview

In Product and Solution Marketing, we have several processes to manage the work the team does

1. Commitment management - how do we know what we're committed to
1. Epics (larger projects) - how do we plan big projects
1. Monitoring and reporting progress - how do we track progress
1. Using Labels and keeping them clean - how do we keep our labels
1. Priority - how do we indicate what's most important
1. Managing Workflow - how do we manage our workflow

Commitment management

We are often asked / requested to work on multiple efforts, across the company. For example,

- An event, needs booth messaging
- A campaign, needs positioning/messaging and perhaps a gated white-paper
- A campaign, needs a customer case study
- A team wants an analyst inquiry

Either way, a real challenge is how to manage and track our commitments to support other workstreams.

We've established the following workflow/process in order for us to consistently capture requests and manage our commitments.

The bottom line: If we don't have a SMRequest Issue that captures our Commitment, then it's invisible and not really a 'commitment'.

SM Request Process

The process is simple:

Here's a short overview of the process:

<iframe width="560" height="315" src="https://www.youtube.com/embed/culHNintg1o" frameborder="0" allow="accelerometer; autoplay; encrypted-media; gyroscope; picture-in-picture" allowfullscreen></iframe>

The process is simple:

1. Anyone can open an SM Support Request Issue this link will the A-SM-Support-Request template.

1. The Product and Solution Marketing leadership team will review the request(Daily), assign it to the ideal SM Team, prioritize the work and plan how to support your requests.

SM Request Process Flow

Strategic marketing request review and assignment flow (note: the label smrequest indicates a request for Product and Solution Marketing support)

1. New requests start with the label smreq::newrequest.

New requests will generally EITHER be assigned to a team and team member or put in the strategic marketing backlog

1. Backlog smreq::backlog for future scheduling, sequencing, and implementation. note: add issue to the SMBacklog milestone for tracking. NOTE: Issues in the backlog are NOT yet committed to be done!

1. Assigned smreq::assigned to team members. When an issue is assigned, it is added to the quarter milestone so we can track status of all the work in flight. NOTE: Assigned issues should be considered committed to be done!

1. Assigned-Queue smreq::assigned::Queue are issues in an individual team member's personal backlog/ queue. They are managing their work flow and this issue is in their queue to add to a sprint/milestone and then deliver.

1. Assigned-In Progress smreq::assigned::InProgress are issues that the team member is actively working. They SHOULD be in a specific milestone.

1. Assigned-Waiting smreq::assigned::Waiting These issues are in progress, but are blocked, waiting for some other contribution in order to progress. (such as waiting for an external approval, a design, draft, engineering, etc.) These issues SHOULD typically be in a milestone, and would typically move to the next milestone if the blocker is not resolved.

1. When complete, the team member will update the issue with smreq::completed and Close the issue

1. Exceptions. There are several cases where an issue is not completed, but should be closed.

1. transferred smreq::transferred for requests that belong in a different team (Field Marketing, Sales, Ops, etc). Once an issue is transferred, it should be closed

1. declined smreq::declined - when an issue is in the backlog and it is no longer relevant or does not make sense anymore. Close the issue when you decline it.

1. canceled smreq::canceled - when an issue is in the backlog and it is no longer relevant or does not make sense anymore. Close the issue when you decline it.

1. Triage Standup. 15 min standup, where the leadership team reviews New Requests to triage them to the most appropriate team. From there, team leads either move to the backlog or assign for immediate work.

Managing the request process

GitLab provides several ways to visualize and manage our work:

1. SM Request Board provides a way to visualize the tracks status of requests grouped by labels. There are several limitations: - lots of scrolling, no sorting, only one action when dragging. So - these different views may be more useful:

1. List Views: enable you to view multiple issues in a common status and you can sort, filter, and make mutiple updates to multiple issues in one step. See the Edit Issue button at the top right.

1. Strategic NEW REQUEST List View This view provides a way to see ALL the New Requests in one list for all of Product and Solution Marketing.

1. Product and Solution Marketing TRIAGE view This view is all of the Triaged issues.

1. Product and Solution Marketing ASSIGNED view This view of all the issues assigned.

1. Product and Solution Marketing Backlog view

1. Assigned and PMM view View shows all the PMM team issues that are currently assigned.

1. Milestones: Milestones give us the ability to track issue completion over time. Milestones also make it easy to look at the related issues based on specific labels and on who the issues are assigned to. Currently, GitLab ONLY allows for one milestone per issue, a limitation that will change in the 12.7 release of GitLab release. In Product and Solution Marketing, we're using two different Milestones to visualize status:

1. Current Quarter Work in progress - this is a PRIMARY VIEW to see what we're working on and the progress toward closure for the overall team. At the end of the quarter, we will create a new milestone for the next quarter and move any unfinished issues to the new

milestone. (and potentially label them as being from a previous milestone.)

1. Product and Solution Marketing Backlog - this is a view of our current backlog which makes it easy to navigate to groups of related issues based on label such as (Priority, team, or usecase)

1. Quick Actions: - Quick actions are commands that you can enter into a comment on an issue and quickly update the status of the issue. Through quick actions you can

- add or remove a label
- assign or unassign the issue to someone
- add or remove from a milestone
- open or close the issues

Quick actions are very,very helpful and efficient when you want to make multiple changes to an issue and have the issue open. Here are a few handy quick actions for Product and Solution Marketing:

Step / Action	Quick Action Code
Triage for the PMM team	/Label ~"smreq::triage" ~pmm
Triage for the Tech PMM team	/Label ~"smreq::triage" ~techpmm
Triage for the Partner Marketing team	/Label ~"smreq::triage" ~"Partner Marketing"
Triage for the Competitive Intel team	/Label ~"smreq::triage" ~"Competitive Intelligence"
Triage for Market Research/Customer Ref Team	/Label ~"smreq::triage" ~mrci
Moving to backlog	/Label ~"smreq::backlog" /Milestone %"SM - Backlog"
Assigning to a team member	/Label ~"smreq::assigned" ~"status::wip" /Milestone %Q4FY20 /Assign @<TeamMember>
Completing an issue	/Label ~"smreq::completed" /close

SM Request insights

We are actively working to leverage GitLab insights in order to monitor how the process is working, learn, and improve over time.

- SM Request Overall - the whole processes

GitLab Product and Solution Marketing PMM Insights

Epics and Milestones - planning and tracking our work

Regular Work

- Milestones for each quarter to track progress of 'assigned' work

For example, in order to visualize all our regular work in a given quarter, we have a "Quarter Milestone" that makes it possible to visualize and summarize all the work within a given quarter. This is an experiment to decide how to best use milestones in our regular work. In the near future (12.10 or 13.0), GitLab will support assigning Multiple milestones to a given issue, which will open the door to both managing real "Sprints", and other topics through the Milestone feature in GitLab. (Today - the limit is 1 Milestone per Issue)

The first time we applied a milestone to regular work was in Q4-FY20, where we saw the pattern of new work flowing in, while other work was completed and closed.

In Q1-FY21, we are continuing to use a milestone to track regular work, and as we learn about our patterns and flow, we believe we will be able to increase our velocity and flow.

As of 13 April:

- 332 total issues
- 173 open and
- 159 closed

Complex projects

When we have large and complex projects, we manage the work through:

- Epics - Define major initiatives with inter-related work streams
- Sub Epics - Sub work streams for different components of a major initiative
- Issues - Specific deliverables / outputs
- Milestones - define time windows to track completion of deliverables.

For example the UseCase GTM Project to build out the messaging, demos, comparisons, case studies and proof points for the Use Cases. Here specific the Epic for a given use case is broken down into sub epics, and then issues are created and associated with the correct epic.

- An Overall Epic - UseCase GTM Epic
- and then child Epics for each UseCase - for example:
 - SCM UseCase Epic
 - CI UseCase Epic
- And then we broke each epic into Months so we could see the associated
 - SCM Month 1 Epic

<iframe width="560" height="315" src="https://www.youtube.com/embed/D74xKFNw8vg" frameborder="0" allow="accelerometer; autoplay; encrypted-media; gyroscope; picture-in-picture" allowfullscreen></iframe>

We've organized our UseCase GTM work by month, and have a Monthly "Sprint"/Milestone that helps

us track completion of the issues/deliverables.

For example this "Milestone" - shows a summary of ALL the usecase work in the Month of April.

Here is a link to the current UseCase 2020-3 milestone.

Workflow - Refine, Sprint, Retrospective

The Product and Solution Marketing team uses a 2-week sprint model, scoping and weighting work items (Issues) to produce the greatest increment of value at the end of each sprint. While our process is similar in many regards to Scrum, we do not practice Scrum Events as separate ceremonies, instead working them into our existing meeting workflow.

We plan, manage, and track our work in the Product Marketing - Overview Issue Board.

On an ongoing basis, individual DRIs update the status of all active items that are assigned to them as the DRI in the Product Marketing backlog. Before the end of one Sprint, they will estimate which items are most appropriate for the following Sprint (Sprint Planning) and confirm this Sprint Backlog with their manager in a weekly 1:1 meeting.

Refine

In order to plan future work it is important to ensure that the request is a) Clearly written and unambiguous, b) What "Done" for the issue is clearly defined. b) The issue/work is free of constraints, and d) the request can be completed in a 2 week sprint. Issues are "Refined" when these questions have been addressed.

Refinement Status

Where ANY issue that has not been refined (DRI, clarity, completable, unconstrained, achievable, prioritized) would start with "unrefined"

- refine::unrefined - The issue is not ready to be worked
- refine::refined - The issue is ready

Refinement Standards

The first step is to identify a DRI. The DRI then owns checking the remaining boxes for the issue.

yaml

Refinement

- [] DRI: Has a DRI been identified and do they accept responsibility?

- [] Clear: Are the expectations and work required unambiguous and clearly defined?
- [] Completable: Is there a clearly-stated definition of done?
- [] Unconstrained: Is the work free of blockers and dependencies? If not, address blockers first or leave in Product Backlog until blockers are removed.
- [] Achievable: Is the issue something that can be completed within a 2-week timebox? If not, decompose it.

set the refined label

Prioritization

Has the value of the feature been assessed based on input from key stakeholders?

Priority

- priority::1
- priority::2
- priority::3

Once an Issue has been labeled 'defined' and assigned a 'priority' it is ready for inclusion in a sprint.

Sprint

The goal of the 2 weeks sprint is to agree on what can be completed in the sprint, and commit to delivery. The goal is to limit adding new work in progress and focus on delivering the committed scope of work in the sprint. Urgent and unexpected stuff happens, so we will always need to have flexibility in our approach and capacity on our team to respond to short notice changes. But, the general pattern should be that we don't add scope to the sprint, unless all the work is completed.

To provide insight and clarity on status we will leverage Issue/Epic Health Status on priority issues.

At the beginning of the Sprint, DRIs will assign the 'On Track' status to agreed-upon priority issues. As work progresses, anyone contributing to the work should update the Health Status as appropriate to surface risk or concerns as quickly as possible, and to jumpstart collaboration on getting an issue back to "On Track".

Retrospective

After each sprint, the team should reflect and document learnings about what worked, what didn't work and how to improve.

Asynchronously, we will document what we learn in this retrospective document and improve.

Metrics and KPIs (GitLab Insights)

We are experimenting how to utilize GitLab Insights

For example, one experiment in Product Marketing is tagging our work based on specific outputs / domain. We're using scoped labels "pmm::xyz" to tag issues based on the type of output and objective:

- pmm::AR Analyst Relations (briefing, inquiry, and research)
- pmm::collateral Developing collateral such as white papers, data sheets, etc.
- pmm::Deck Developing slides and presentations
- pmm::Enable Developing and delivering enablement (mainly to the field)
- pmm::Events Developing and delivering content at events (online and in person)
- pmm::messaging Developing positioning and messaging
- pmm::PR Briefing and updating press and media
- pmm::Research Planing and conducting market research
- pmm::Sales Direct support of sales with customers
- pmm::Web Developing content for web pages (blogs, web pages, etc)
- pmm::other Other work that doesn't fit above

- Using the GitLab triage bot, we can automatically assign the work into a different set of scoped labels "pmM::External" or "pmM::Internal". Where "External" means it directly is related to engaging prospects and customers, building and accelerating pipeline. And "Internal" indirectly helps us grow and improve.

Through this, we can track our work and improve our balance and focus:

PMM Insights - (Internal vs External)

PMM Insights (External details)

PMM Insights (Details)

GitLab Product and Solution Marketing PMM Insights

```
<iframe width="560" height="315" src="https://www.youtube.com/embed/OMTfPsLa98I"
frameborder="0" allow="accelerometer; autoplay; encrypted-media; gyroscope; picture-in-picture"
allowfullscreen></iframe>
```

Labels and Label Hygiene

We have adopted the GitLab Triage bot as a way to establish clear policies for labels and issue hygiene. This allows us to create a set of process rules and policies and then automatically apply them to our issues. This helps us to keep issues in the expected state with the expected labels.

See this summary of how to set up and use the GitLab Triage Bot

<iframe width="560" height="315" src="https://www.youtube.com/embed/Tp79e5sgpao?start=1" frameborder="0" allow="accelerometer; autoplay; encrypted-media; gyroscope; picture-in-picture" allowfullscreen></iframe>

Priority and Prioritization

At this point, we have three "Priority" labels in Product and Solution Marketing. These are both scoped labels (so you can't have both priority::1 and priority::2) at the same time. The labels are also defined as "Priority", so they will sort issues where they are assigned.

- 1. priority::1
- 1. priority::2
- 1. priority::3

Over time, we will be establishing guidelines about how we consistently use these labels to communicate priority within the team.

Example of managing our workflow

Some of us in the team use GitLab Issue Boards to manage our workflow. Using the issue board and scoped labels such as Open, To-Do, Doing, Waiting, Closed, the issue board gives us a visual representation of issues assigned to us.

- 1. Label Closed : Issues that have already been completed and closed
- 1. Label Waiting : Issues that are waiting for input from other teams
- 1. Label Doing : Issues that we are currently actively working on
- 1. Label To-Do : Issues that we will pick up next
- 1. Label Open : Issues in our backlog

We move issues across these stages based on the progress and order them within the stage based on our priority of working on them. This helps team members to manage the issues assigned to them better as well as managers to asynchronously get a view of what's in progress and what's blocked.

SECTION: marketing/brand-and-product-marketing/product-and-solution-marketing/it-groups/_index.md

Hidden IT Groups

What is a "Hidden IT Group"?

Within IT organizations, you will find there is a formal organization chart, with defined roles and responsibilities. These often form around functional areas like IT Operations, Application Development, Security or Portfolio Management, or Service Management. There are also hidden and invisible 'groups' that are defined by the traits and characteristics that the people share. While these groups might align to a formal organization in the org chart, they

might not.

Why are hidden groups relevant

When you consider modern organizations and culture, often we organize into groups that share common values, culture, processes, language, standards, and norms. Understanding the culture and values of these groups is incredibly important when we try to connect with them and offer solutions to alleviate their pain points.

Trait	Description
Values	Some people may value innovation, while others value quality
Culture	How people work may be different
Challenges	They will perceive different challenges in their work
Thought leaders, analysts, & conferences	They will follow and learn from different leaders
Tools	the tools they choose will meet their specific needs
Processes, practices, and standards	ITIL, CMMI, Agile, PMI, etc
Conflicts	They may see things differently from other 'it groups'

IT Groups in enterprise IT/technology

In IT, there are several 'hidden groups' that describe groups of people and their shared values and common traits. While there are many possible 'IT Groups' to explore, the goal here is to offer a simple framework of the most typical and common divisions in order to understand their needs, goals, challenges and language.

Groups vs Personas

- Groups are Not personas. Rather 'personas' are found inside of different groups. Think of these IT groups as a collection of different roles and personas.
- Groups are Not (necessarily) Organizations, but many places might have teams defined closely to these 'Group' boundaries.

Diagram illustrating the relationship between IT groups and organizational functions:

- Hidden IT groups
- Build / Develop
- Test
- Run
- Protect

Organizational Functions:

- Organize
- Build / Develop
- Test
- Run
- Protect

--|-----

-----|

| Values | On time, on budget, on track. Organization. Keeping the team focused on delivery for the business.
| Innovate and deliver cool new solutions as fast as possible, help the business compete with the best solutions to meet their needs. |
Quality is key, finding defects, protect users have a bad experience
| Uptime, performance, availability, efficiency. Keep it running with the lowest cost.
Restore service as fast as possible. Avoid and recover from outages (disasters)
| Manage and minimize risks, protect our systems, data and business from cyber threats everywhere, inside and outside.
|
| Culture | Complex projects and programs, securing teams in a matrixed world to organize and deliver. Agile makes it difficult to predict future delivery.
| Collaborative and creative. Early adopters of new technology.
| Processes and traceability to double check that each release is suitable for production. Often view their role as guardians, protecting users from defects.
| Process oriented, often based on ITIL processes.
| People, process, technologies are a constant balancing act. The goal is to have enough of each. You can't be 100% secure, but need to have a layered approach to security. While security does introduce friction, the goal is to enable the business.
|
| Typical titles | Project Manager, Scrum Master, Product manager, Business Analyst
| Developer, architect, DBA, DevOps Lead, DevOps Engineer, DevTest
| QA Lead, Tester, Performance tester, Automation Tester, DevOps Lead?, DevOps Engineer, DevTest
| Sys Admin, Perf Monitoring, Incident Mgt, Release Mgr, SRE, DevOps Lead, DevOps Engineer
| Security Operations, Security Analyst, Application Security, Penetration Tester, others
|
| Hidden IT Groups | Organize
| Build / Develop
| Test
| Ops
| Protect
|
| Tools they use | PPM, Agile Portfolio Mgt, Project Management, Value Stream mgt, Requirements Mgt,
| SCM, Code Review, Code Quality, API Mgt, Architecture Mgt, Binary Repository, CI
| ALM, Quality Mgt, Test Automation, Perf Testing
| CD, Container Mgt, Cloud Mgt, ITSM, Service Desk, Incident Mgt, APM,
| App Security, Web App Firewall, SEIM?, SW Composition Mgt,
|
| Process | PMI, PMBOK, Prince2, SAFE, Earned Value, WBS
| CI, Agile, RUP, Pair Programming,
| Risk Based Testing, Traceability, TMMI, CMMI
| ITIL, Release Mgt, Change Mgt, Incident Mgt, Service Management
| COBIT, ISO
|

Challenges	Complex projects and programs, securing teams in a matrixed world to organize and deliver. Agile makes it difficult to predict future delivery.
Contributing to multiple projects, workstreams and systems. Supporting production and new development. Burdensome processes get in the way of delivering value.	
Complicated software changes that is hard to accurately test. Test environments that don't match production and keeping up with the rate of change from Dev	
Managing complex legacy infrastructure where business expects 24x7 at no cost, while developers want to make changes and break things. Stuff breaks for no reasons outside of your control.	Expected to
protect everything, but rarely involved in projects early or often enough. Frequently blamed for project delays and rework. Often late in SDLC, an isolated team, not included in developing new requirements, testing etc. From an operational standpoint, signal fatigue is a real problem.	
Utopia	Able to quickly prioritize business demand, initiate projects and organize resources to deliver on time and on budget. Visibility into execution helps to solve issues early. Able to focus on innovation and solving business problems. Responsive to change and not burdened by bureaucratic processes. They care about their software and the business value they deliver. Testable requirements and testable applications enable automated testing of every change identifies defects, enabling the QA team to focus on exploratory testing and edge cases. Changes never break SLAs, when problems happen, MTTR is rapid. Production infrastructure is easy to manage, scalable, efficient and available to support business demand. Self service enables day to day work and there are no snowflakes. Because security is never 100%, ideally, we would have both proactive and reactive capabilities. For example, application security and shifting left would be proactive measures, along with secure SDLC training for developers. On the reactive side, we have security operations and red teaming capabilities that catch what wasn't discovered earlier in the process. All of these capabilities need to be driven and governed by policy and process, and adequate technologies need to be deployed to ensure success.
Hidden IT Groups	Organize
Build / Develop	
Test	
Ops	
Protect	
GitLab Stage Mapping	Manage, Plan
Create, part of Verify	
Verify, part of Plan	
Package, Release, Configure, Monitor	
Secure, Protect	

- What about DevOps?

DevOps is a movement, bringing teams together to work across their group differences.

- What about Stages?

These hidden IT groups relate to one or many stages. See the above table.

- Are there other 'hidden groups'?

The closer you look at any of these macro groups, you might see 'sub groups' with in them. For example in the Run group, you might discover a service faction or an infrastructure/network faction. Or in the QA/Test group, you may find performance testing and devtest factions.

SECTION: marketing/brand-and-product-marketing/product-and-solution-marketing/it-groups/organize.md

Overview

People who are part of the Organize IT Group are typically project, program and portfolio managers who are responsible for organizing, planning, and executing large projects and initiatives. They are focused on building the team, defining the tasks and work, and then delivering on the commitments.

Values

On time, on budget, on track. Organization. Keeping the team focused on delivery for the business.

Culture

Complex projects and programs, securing teams in a matrixed world to organize and deliver. Agile makes it difficult to predict future delivery.

Personas / job titles

Clearly these titles:

- Contributors: Project Manager, Scrum Master, Product Manager(see note about Product Manager), Business Analyst (BA)
- Managers: Program Manager, Portfolio Director
- Executives: PMO Director, EPMO Director, perhaps VP EPMO or VP Shared Services.
- Note: Product Manager is an emerging role within IT organizations. We expect that many of the people will migrate from traditional Project Manager roles, however the scope of the role is dramatically broad and will likely also see a wide variety of people filling this role. For now, we're listing it in this hidden IT Group.

Challenges

Complex projects and programs, securing teams in a matrixed world to organize and deliver. Agile makes it difficult to predict future delivery.

Ideal world

Able to quickly prioritize business demand, initiate projects and organize resources to deliver on time and on budget. Visibility into execution helps to solve issues early.

SECTION: marketing/brand-and-product-marketing/product-and-solution-marketing/it-groups/protect.md

Overview

People who are part of the Protect IT Group are typically information security and compliance professionals who are responsible to minimize business risk in IT systems. They establish and maintain policies, processes and procedures to help ensure application changes are secure and regularly evaluate IT systems to identify vulnerabilities.

Values

Manage and minimize risks, protect our systems, data and business from cyber threats everywhere, inside and outside.

Culture

People, process, technologies are a constant balancing act. The goal is to have enough of each. You can't be 100% secure, but need to have a layered approach to security. While security does introduce friction, the goal is to enable the business.

Personas / job titles

Security Operations, Security Analyst, Application Security, Penetration Tester, others

Challenges

Expected to protect everything, but rarely involved in projects early or often enough. Frequently blamed for project delays and rework. Often late in SDLC, an isolated team, not included in developing new requirements, testing etc. From an operational standpoint, signal fatigue is a real problem.

Ideal world

Because security is never 100%, ideally, we would have both proactive and reactive capabilities. For example, application security and shifting left would be proactive measures, along with secure SDLC training for developers. On the reactive side, we have security operations and red teaming capabilities that catch what wasn't discovered earlier in the process. All of these capabilities need to be driven and governed by policy and process, and adequate technologies need to be deployed to ensure success.

SECTION: marketing/brand-and-product-marketing/product-and-solution-marketing/it-groups/run.md

Overview

People who are part of the Run IT Group are typically information operations professionals who

are responsible for the day to day operation and efficiency of the organization's IT systems.

Values

Uptime, performance, availability, efficiency. Keep it running with the lowest cost. Restore service as fast as